

IGXSecure — Git Command Reference

Project: IGXSecure

Repo: <https://github.com/108ravirajsinh/IGXSecure>

Branch: main

1. Daily Workflow

Check what changed before doing anything

```
git status
```

Shows: modified files, new untracked files, staged files.

See full change history

```
git log --oneline
```

Example output:

```
88308c1 feat: Phase 1 + Phase 2 - static UI, Express backend  
a1b2c3d chore: Phase 0 - scaffold project structure
```

Pull latest from GitHub before starting work

```
git pull origin main
```

Always do this at the start of each session to stay in sync.

2. Saving Your Work (Stage → Commit → Push)

Stage all changed files

```
git add .
```

Stage a specific file only

```
git add backend/server.js
```

Commit staged files with a message

```
git commit -m "feat: add health endpoint to backend"
```

Push to GitHub

```
git push origin main
```

Do all three in one flow

```
git add .  
git commit -m "your message here"  
git push origin main
```

3. Commit Message Convention

Use a consistent prefix so the history is readable:

Prefi x	When to use	Example
feat:	New feature or page	feat: add dashboard stories panel
fix:	Bug fix	fix: cors header not returned
chore:	Setup, config, cleanup	chore: update docker-compose ports
style:	CSS/UI changes only	style: update landing page button color
docs:	README, markdown files	docs: update project plan phase 3
refactor:	Code restructure, no new feature	refactor: split security config into own file
phase :	Phase completion	phase: Phase 2 complete - Express backend

4. Undoing Mistakes

Undo changes to a file (not yet staged)

```
git checkout -- backend/server.js
```

Restores the file to the last committed version.

Unstage a file (staged but not committed)

```
git restore --staged backend/server.js
```

Undo the last commit but keep your changes

```
git reset --soft HEAD~1
```

Your files stay changed — only the commit is removed.

See exactly what changed in a file

```
git diff backend/server.js
```

5. Branches (for safe Phase development)

Create a new branch for a phase

```
git checkout -b phase-3-nginx
```

Switch between branches

```
git checkout main  
git checkout phase-3-nginx
```

Merge a phase branch into main when done

```
git checkout main  
git merge phase-3-nginx  
git push origin main
```

Delete a branch after merging

```
git branch -d phase-3-nginx
```

See all branches

```
git branch -a
```

6. Remote (GitHub) Management

Check where your remote points

```
git remote -v
```

Expected output:

```
origin https://github.com/108ravirajsinh/IGXSecure.git (fetch)
origin https://github.com/108ravirajsinh/IGXSecure.git (push)
```

Update remote URL (if repo is renamed)

```
git remote set-url origin https://github.com/108ravirajsinh/IGXSecure.git
```

Pull with unrelated history (first-time merge fix)

```
git pull origin main --allow-unrelated-histories
```

7. Viewing History

See last 5 commits

```
git log --oneline -5
```

See who changed what line in a file

```
git blame backend/server.js
```

See changes in the last commit

```
git show HEAD
```

See changes between two commits

```
git diff a1b2c3d 88308c1
```

8. .gitignore — What Must Never Be Committed

These are already in your .gitignore:

```
.env          ← your TOKEN_SECRET and API keys
node_modules/ ← auto-generated, too large
*.log         ← server logs
*.pem / *.key ← SSL certificates
```

Check if a file is being ignored

```
git check-ignore -v backend/.env
```

Force-remove a file from Git tracking (but keep it on disk)

```
git rm --cached backend/.env
git commit -m "chore: remove .env from tracking"
```

Use this if you accidentally committed a secret file.

9. Phase Completion Flow

At the end of every phase, run this full flow:

```
git add .  
git status          # review what's being committed  
git commit -m "phase: Phase X complete - description"  
git push origin main
```

Then tag it for easy reference:

```
git tag phase-2-complete  
git push origin --tags
```

10. Quick Reference Card

Task	Command
Check status	git status
Pull latest	git pull origin main
Stage all	git add .
Commit	git commit -m "message"
Push	git push origin main
View history	git log --oneline
Undo unstaged change	git checkout -- filename
Unstage file	git restore --staged filename
New branch	git checkout -b branch-name
Switch branch	git checkout main
Merge branch	git merge branch-name
Tag a phase	git tag phase-X-complete
Push tags	git push origin --tags